

Balancebot



Lab Goals

ACTING

- Implementation of odometric navigation
- Dynamic control of DC motors to balance and control a wheeled inverted pendulum
- Trajectory Following for a mobile robot

SENSING

- Use of wheel encoders for performing odometry
- Use of MEMS IMU sensors to determine orientation (heading, pitch angle)

INFRASTRUCTURE

- Build a system from the ground up based on an embedded system
- Use of Beaglebone with Robot Control Library in C for soft real-time control

Overview

In this project, you will be working with a toolkit for building a Mobile Robotics Cape for the Balancebot. The main goal is to integrate the Mobile Robotics Cape with the Beaglebone Green, to implement a gyrodometric-based dead-reckoning navigation system, and to control the Balancebot using sensor feedback so it can balance upright even when disturbed (i.e. someone pushes it) or when traversing a commanded path. You will also use potential field planning to navigate a set of waypoints as fast as possible using feedback from the Optitrack motion capture system.

Your final project will consist of a competition on Monday, February 2nd, and a 6 page lab report, due by 11:59PM on Sunday, February 9th.

The Balancebot project is a team assignment. Your team must complete it together through collective participation on each component.

Hardware

Beaglebone Green

1GHz AM335x ARM Cortex A8	3x hardware quadrature decoders
512MB DDR3 RAM	3x 2 channel hardware PWMs
4GB eMMC on-board flash storage	Analog: 7 channel 12bit ADC
NEON floating-point accelerator	Serial: 2 i2c, 2 SPI, 5 UARTs
2x PRU 32-bit microcontrollers	Most pins can be GPIO
USB 2.0 host & 100Mb Ethernet	\$45

Connecting to the BBB via USB cable:

- 1) Attach to Linux laptop via USB cable
- 2) Wait for the laptop to report that it is connected to ethernet (where the wifi symbol is shown)
- 3) Open a terminal window and ssh into the BBB with user name 'debian'

```
$ ssh debian@192.168.7.2
```

The debian account password is `temppwd`. Please **DO NOT** change this default password. You should now see a new prompt in the terminal window indicating you have successfully logged in.

If you are having problems connecting to the Beaglebone, you may need to define a static IP for your computer on the correct interface (you will find 2, that's a 50/50 chance)

IP: 192.168.7.1

Netmask: 255.255.255.0

In Ubuntu set the network interface to only use this interface for resources on its own network. Once you have logged in over USB, you can setup a wifi connection which is slower, and a bit less reliable, but will allow you to run everything without the USB cable plugged in.

Setting the SSID on the Wifi Access Point

The Beaglebone green on the Balancebot is running hostapd in order to provide a WiFi access point. You can change the SSID of the access point by editing the file `/etc/hostapd/hostapd.conf` and changing the line that begins with `'ssid='`. You should change the ssid to `BBG-00XX` where `XX` is your team number.

Connecting to the BBG via wireless:

- 1) Power up the BBG board with USB cable or 5 volts power supplier
- 2) insert or reinsert the WiFi dongle and ensure the blue light inside is on.
- 3) Connect to wireless network "BBG-XXXX" where XXXX is the number you set up before
- 4) if the SSID doesn't show up, unplug and replug the wifi dongle.
- 5) SSH into BBB with "ssh debian@192.168.4.1"

Turning off or rebooting

If you want to shutdown or restart the BBB, you should issue either the Linux `halt` or `reboot` commands. These commands must be run as root, so preface each command with `sudo`. The reset button (on the top right in the picture above) should only be used as a last resort.

```
$ sudo halt  
or  
$ sudo reboot
```

Moving files

In order to copy files to (and from) the Beaglebone, the command `scp` is useful. Connected with USB the command would be:

```
$ scp -r PATH/TO/SOURCE debian@192.168.7.2:PATH/TO/DESTINATION
```

The `-r` flag allows you to copy recursively, i.e. all folders and files starting at the source directory. If the files on the destination already exist, they will be overwritten. Any code you write or any files you create on the Beaglebone you should back up to your own laptop for security. Beaglebones have been known to die if not used carefully. While we hope this will be rare, it is a definite possibility. To copy files from the Beaglebone to your laptop using the USB cable:

```
$ scp -r debian@192.168.7.2:PATH/TO/SOURCE PATH/TO/DESTINATION
```

An easy way to access the files on the Beaglebone directly from the lab laptop is by connecting to it as a server. In Ubuntu, open a new files window and use File > Connect to Server..., then enter `ssh://debian@192.168.7.2/` or `ssh://debian@192.168.3.XXX/` as the server address (depending on if you are connecting over USB or wifi). NOTE: you'll only want to use this mounted file system for git repo actions and opening files in a text editor. DO NOT COMPILE code from the Beaglebone directly on the lab laptop, as this would build code in an x86 environment, which means it won't work in the ARM environment used on the Beaglebone. To build code on the Beaglebone, ssh in and issue "make" commands from there. Note: Another useful command for copying files over ssh is the `rsync` command. Similar to `git`, `rsync` looks at the differences between the directories and only copies new, or modified files.

Flashing the BBBG with an SD Card

To flash a new BBG with the SD card there are two options, you can use the flasher, or the image.

One SD card we have is a flasher.

- 1) Insert the flasher into a BBG and power on the board
- 2) The BBBlue will boot from the SD card (takes a little while) and then automatically start copying the contents of the SD card onto the internal memory on board the Beaglebone.
- 3) You will see 4 blue lights flashing like a "Cylon"
- 4) When the flashing pattern stops (5-10m) the flashing process is finished
- 5) Remove the SD card and press the Reset button

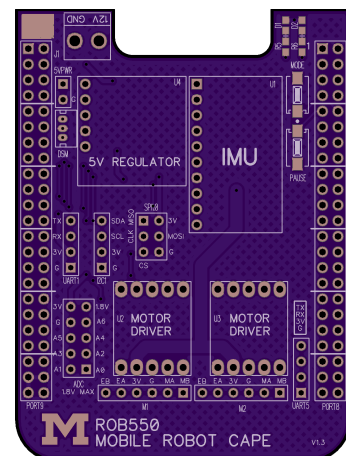
The image SD card is safer and will not automatically run the flasher, so you will have to do it yourself.

- 1) Insert the image SD card into a BBG and power on the board
- 2) The BBG will boot from the SD card (takes a little while)
- 3) Once it is booted, log in over ssh
- 4) Once logged in, run: `sudo /opt/scripts/tools/emmc/init-emmc-flasher-v3.sh`
- 5) You will see 4 blue lights flashing like a "Cylon"
- 6) When the flashing pattern stops (5-10m) the flashing process is finished
- 7) Remove the SD card and press the Reset button

To copy the contents of your beaglebone to a new SD card, you can follow the exact steps above, except boot from the internal memory and then insert an SD card.

Mobile Robotics Cape

The Mobile Robot Cape (MRC) is a custom cape that is compatible with a subset of the functions in `librobotcontrol`. The cape has just the hardware needed to run the balance bot, a voltage regulator, two motor drivers, and an IMU, as well as connection points for the motors and radio receiver. A schematic of the cape is available in



the Schematics folder on Google Drive in the /Balancebot folder. **You will be referring to the provided schematic for finding and filling in your needed info for the first tasks 1.1, 1.2 of the lab.**

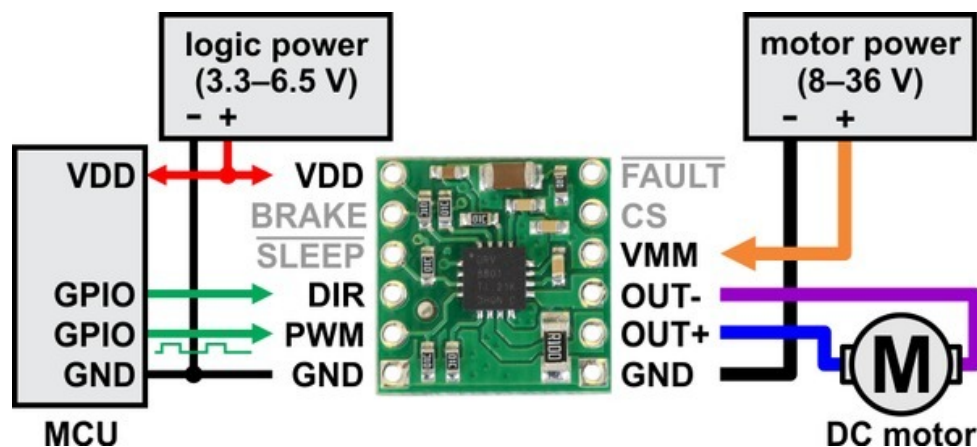
Motors

The provided motors operate on 12V and have a gear ratio of 20.4:1. The motor's datasheet can be found [here](#). Do not trust the manufacturers data for these motors. You should independently test each motor for the parameters you need for the lab. The connectors on the motors have the following pinout:

Color	Function
Red	Motor +
Black	Motor -
Green	Encoder GND
Blue	Encoder VCC (3.3V)
Yellow	Encoder A
White	Encoder B

Motor Drivers

The motor driver PCB is attached to the MRC and connected as in the below schematic. In addition to the PWM, and DIR pins, the BRAKE pin is connected to a GPIO pin on the Beaglebone. The motor driver's product page can be found [here](#). The CS (current sense) pin of each motor driver on the MRC is connected to an analog input. By reading this analog input you can determine how much current the motor is drawing.



Encoders

The motors have a magnetic quadrature encoder on the motor shaft. These encoders can be read directly by the beaglebone to determine the position or velocity of the wheel. The encoders have a resolution of 48 transitions per motor shaft revolution. The magnetic hall effect sensors require 3.3V and Gnd.

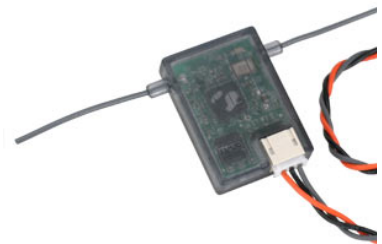


MPU9250 IMU

The MPU9250 is a 9DOF IMU with 3-axis accelerometer, 3-axis rate gyroscope, and 3-axis magnetometer. A big advantage of the MPU9250 over other IMUs is the inclusion of the Digital Motion Processor, a separate low power processor which performs sensor fusion algorithms on board. From this processor you can read orientation quaternions or Tait-Bryan angles (roll, pitch, yaw) directly. You can write your own data fusion filter as well. A good resource on data fusion filters for IMUs is [here](#).

DSM Receiver

The DSM Satellite receiver will bind to only one transmitter at a time, and communicates the positions of the sticks on the transmitter over a serial connection to the beaglebone. To bind the receiver to a transmitter:



1. Turn off the transmitter
2. Run the `rc_bind_dsm` program and select option 3.
3. if option 3 does not work, try option 1.
4. Unplug and replug the satellite receiver from the cape.
The orange light should blink rapidly.
5. While holding up the bind button or switch on the back left of the transmitter, power on the transmitter. The LCD screen should read BIND, and the orange light on the receiver will blink slower than before. The transmitter will continue to start up and when completed, the orange LED on the receiver will be on continuously. Now the transmitter and receiver are bound together, and will remain bound until the receiver is rebound to another transmitter.

Now, whenever the radio transmitter is detected by the receiver the orange light will be ON, even after power cycling. Run the `rc_test_dsm` program to test your radio link. Run the `rc_calibrate_dsm` program to calibrate the min and max of the transmitter.

The DXe transmitter has 6 channels, and can be configured however you like. You will need three channels: one for steering, one for moving, and one for switching between manual and autonomous mode. You can set up the switches on the transmitter to perform other functions if you wish. You will be sharing a transmitter with the other group at your bench, so you may want to add a model to save all your settings too. You can move this model to other transmitters with an SD card.

Software

libroboticscape

The Robot Control Library is already installed on your Beaglebone, and the source code is in the folder `~/Install/librobotcontrol`. Check out the examples folder, the [website](#), or the [github repository](#) for how to use the functionality it provides.

There is an extensive list of [modules](#) provided by the library, including functions for:

- linear algebra
- matrix and vector manipulation
- digital filters (including PID and Kalman filters)
- digital I/O
- serial communications
- esc and servo control
- reading the onboard imu & barometer
- pthreads for setting up real-time threads
- time functions for precision sleeping and timestamps

The example and utility programs provided by Robot Control Library are installed on the BBG and can be run from the command line. The listing of programs is available [here](#).

GPIO

When setting up and controlling GPIO pins you must know the chip and pin number on that chip for the GPIO pin. On the beaglebone there are 4 gpio chips, each with 32 pins and the pin names don't correspond to the physical header pin on the beaglebone. When initializing the pin, you must also indicate what function you want the pin to have. Pins can be both inputs and outputs, but can also be active low or not (you write a 1 and it goes low), and can be open drain or open source (which controls whether the pin can be used to source or sink current.) To toggle high and low gpio2.20 at 1s intervals you might do the following:

```
rc_gpio_init(2,20,GPIOHANDLE_REQUEST_OUTPUT);
while(1){
    rc_gpio_set_value(2,20, HIGH);
    rc_nanosleep(1E9);
    rc_gpio_set_value(2,20, LOW);
    rc_nanosleep(1E9);
}
```

PWM

PWMs on the beaglebone are organized in pairs. For each of the 3 subsystems [0,1,2], there are 2 channels, A and B. Each channel can have a different duty cycle, but the period is shared. You must initialize the period of the subsystem first, and then set the duty cycle of the channel. On the Mobile Robotics Cape the PWMs to the motor drivers are channels A and B on subsystem 1. To setup the pwms and set the motor speed to ½ the max speed, you might do the following:

```
rc_pwm_init(1,PWM_FREQ);  
:  
rc_pwm_set_duty(1, 'A', 0.5);  
rc_pwm_set_duty(1, 'B', 0.5);
```

Measure IMU

The Measure IMU template is a bare-bones template for writing code to acquire data from the IMU to take data to help you determine physical parameters of your Balancebot.

Measure Motors

The Measure Motors template is a bare-bones template for writing code to acquire data from the motor current sensors and encoders to take data to help you determine motor parameters.

Balancebot

The Balancebot template is a starting point for your code. There are common files you will need to complete in the common folder, along with the main code `balancebot/balancebot.c`. There is also a completed `test_motors.c` program that will test your implementation of the motor code.

The `balancebot.c` code starts two threads, and attaches the `balancebot_controller()` function to the IMU interrupt. One thread slowly prints data to the terminal, you may add data to print here. Note how the data line is printed over and over again with a carriage return, but no new line, so it appears to just update in place. The second thread is the setpoint thread that reads the data coming from the RC transmitter/receiver pair. You should use one of the switches on the transmitter to control if you are in manual driving mode or autonomous driving mode.

Lab Assignment

Part 1: Getting Started & System Modeling

In this section you will need to determine certain parameters of the system to plug into the provided code and Matlab files for help modeling the system and tuning your inner loop controller.

1.1: Read through the Balancebot reference material and find the answers to the following questions. Use your answers to fill in the relevant parameters in the `common/mb_defs.h` and `common/mb_motor.h` files.

- what pwm subsystem and channels are used to interface the motor?
- what pins (and chips) are used to control the two motor drives?
- what analog channels are used to sense the current for the motors?
- what are the polarities of the motors and encoders ?
- what is the gear ratio of the motor?
- what is the resolution of the encoder?
- what is the nominal wheel diameter?
- what is the nominal wheelbase?

1.2: Using the functions for gpio and pwm in the Robot Control Lib SDK, implement the functions in `mb_motor.c` to enable driving the motors. Test your implementation with the `test_motors` program.

***** DO NOT USE ANY `rc_motor()` FUNCTIONS IN YOUR `mb_motor.c` FILE *****

They are used for different motor drivers and there is a conflict with the BRAKE pin used on our motor drivers. Also using any functions from this library has been known to (for unknown reasons) cause kernel panics.

CHECKPOINT:

- ❖ Demonstrate to an instructor your motors spinning using the `test_motors` program.

1.3: Using the template code in `measure_moments.c`, write a program to collect IMU data, including gyro, accelerometer and Euler (Tait-Bryan) angles. Using this program and the bifilar pendulum procedure below, measure the moments of inertia about the principal axes of your Balancebot. Repeat the experiment a few more times to collect some statistics on the parameters you measure.

REQUIRED FOR REPORT:

- ❖ Report the statistics of the physical measurements you take in a table.

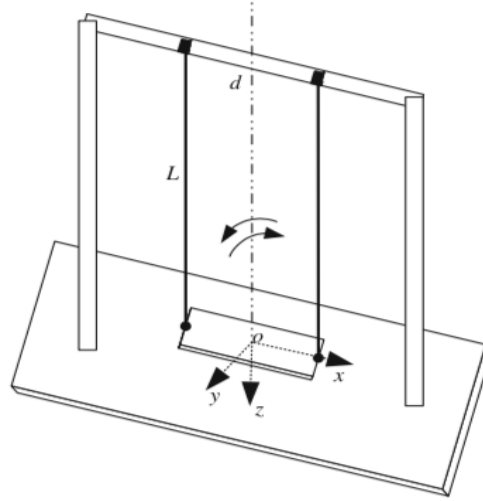


Fig 5. Bifilar Pendulum setup

The distance between the wires is d and the wire length is L , with object levelled. Then, the oz axis of the object and the vertical central line of the wire coincide. The object is rotated manually around oz axis by a small angle. After the object is released, the periodic back-and-forth movement around the oz axis will proceed.

Its swing period is derived as :

$$T_0 = 4\pi \sqrt{\frac{J_{ZZ}L}{m_0gd^2}}$$

Rearranging the terms, the moment of inertia about Z -axis can be determined by the following equation :

$$J_{ZZ} = \frac{m_0gd^2}{16\pi^2L}T_0^2$$

Similarly, orienting the Balancebot such that its body axis is aligned with axis of rotation J_{XX} , J_{YY} can be obtained.

Procedure :

1. Fix two thin wires on the Balancebot body. Adjust two contact points in order to ensure the equal length and the uniform distribution on two sides of the CoG. The moment of inertia about another axis can be measured by changing the position of two contact points.
2. Attach the two thin wires to the fixture separated by the distance so the two wires generate a rectangle shown in the diagram. Record the width d and the length L of this rectangle.
3. Start the data acquisition program and rotate the Balancebot around the suspension axis by a small angle ($<10^\circ$) and release to start the Balancebot oscillating. Analyze the data to determine T_0 , the period of oscillation.
4. Calculate J_{ZZ} using T_0 and substituting it in the equation above.
5. Reorient the vehicle and repeat steps 1-6 to determine J_{XX} , J_{YY} .

1.4: Using the template code in `measure_motors.c`, write a program to take data to allow you to determine critical parameters for the motors. Log data including time, applied voltage, current and speed to help your analysis. Analyze the data (in the program itself or in Matlab or Python) and use other electrical measurements of the motors to determine the following parameters for each motor:

- Motor coil Resistance R
- No Load Speed ω_{NL}
- Motor Constant K
- Stall Torque τ_S
- Friction Coefficients (b & c or just b)
- Inertia of the motor armature, shaft and gearbox I_M

You can use these values to accurately model the motors using the provided MATLAB code.

REQUIRED FOR REPORT:

- ❖ Report the procedure used to measure the motor parameters
- ❖ Report the motor parameters for both motors in a table comparing them to the manufacturer data when available

Part 2: Balance Controller

The goal of this portion of the lab is to create and tune a controller that allows for manual driving of the Balancebot with a high degree of stability. You may implement the final controller you use however you like, though first you will create 2 versions of the controller: A classical controller using successive loop closure (SLC) and a linear controller using full state feedback. In class we will describe a PID controller tuned by looking at step response, and a full state feedback controller with gain matrix found using the Linear Quadratic Regulator method. For controllers using SLC, the inner loop of the controller will control the body angle of the balancebot, while the outer loop will control the body position or velocity. A good resource for the classical controls and state-space modeling in Matlab is available [here](#). An excellent free book on Linear Control Theory is [here](#).

2.1: Implement the inner loop controller in the balancebot template code. The input to your controller should be a reference body angle (NOTE: this may not be 0, you may need a constant offset). The output will either be commands to the motor drivers or torques commanded to your motor controller (which will then output a PWM signal to the motor drivers)

REQUIRED FOR REPORT:

- Describe the controller you implemented using equations.
- Describe the procedure or strategy you used when tuning the inner loop
- Include all relevant parameters of your controller in a table.
- Include a plot of the measured step response of your Balancebot along with the simulated step response of your model for both a large and small step. If your model response does not match your balancebots, discuss why in the report.

2.2: Implement the second loop in your controller. The input should be a reference wheel position and the output should be a body angle to input to your inner loop.

REQUIRED FOR REPORT:

- Describe the controller you implemented using equations.
- Describe the procedure or strategy you used when tuning the outer loop
- Include all relevant parameters of your controller in a table.
- Include a plot of the measured step response of your Balancebot along with the simulated step response of your model. If your model response does not match your balancebots, discuss why in the report.

CHECKPOINT:

- Demonstrate to an instructor your functioning balance controller, even if it needs more tuning.

Part 3: Manual & Heading Control

A heading controller can be implemented simply by sending a differential motor command in parallel with the commands from your balance controller.

3.1: Implement a turning controller (your choice) to change the heading of the Balancebot. The simplest controller can adjust a differential motor speed proportionally to a commanded turn velocity. This controller can run in parallel to your other controller, and the two can be added together.

REQUIRED FOR REPORT:

- Include a plot of the step response of the balancebot heading to a change in heading. You do not need a model response.
- Describe the controller you implemented using equations and a table of relevant parameters.

3.2: Implement RC control in order to manually control the Balancebot. You should set up the controls so that the right stick controls body velocity with forward and back motion, and the left stick controls turning velocity with sideways motion. **You may want to do this before tuning your controllers so you have an easy way to test the behavior of changing gains.**

CHECKPOINT:

- Allow an instructor to drive your robot using the RC controller

Part 4: Odometry & Motion Control

In this section you will develop odometric “dead reckoning” functionality on your balancebot. This code can be carried over to the next lab, so ensure your implementation is general and robust. You can attach a caster to your balancebot and turn off the balancing controller to test your odometry implementation.

4.1: Implement the odometry functions in mb_odometry.c to enable dead reckoning with wheel encoders only. First, test your implementation by manually moving the Balancebot known distances and turning known angles, then use the UMBark procedure to calculate parameters needed to correct systematic errors in your odometry.

REQUIRED FOR REPORT:

- Describe how you validated your odometry model and report the values of correction parameters.

4.2: Either implement the “gyrodometry” algorithm discussed in lecture or implement your own algorithm for fusing the gyro data with odometry data for estimating the heading of the balancebot.

Note: If you feel like you can completely ignore the IMU for heading information or alternatively completely ignore the odometry you should take data to justify your decision.

REQUIRED FOR REPORT:

- Explicitly report the equations used to perform odometry and gyro fused odometry.
- Report all parameters in your odometry and gyrodometry model and how they were determined.

4.3: Using the odometry data, create two high level controllers. The first should allow you to drive arbitrary distances, the second should hold a heading. These controllers should assist you in following a waypoint plan or gradient field.

REQUIRED FOR REPORT:

- Describe the controller you implemented using equations and a table of relevant parameters
- Include a block diagram of the full control system.

4.4: Implement one of the trajectory following algorithms discussed in class. Have your Balancebot drive around a predetermined square path four times with odometry as feedback

REQUIRED FOR REPORT:

- Discuss your trajectory following algorithm.
- Include a plot of your balancebot position from odometry as it follows the trajectory given in class

CHECKPOINT:

- Demonstrate to an instructor an attempt of a single lap of drive square

Competition

Event 1: Balancebot on the mark (200 points)

- ❖ 100 points for balancing within 10cm of a mark for 20s
- ❖ 100 points for balancing and returning to the mark while rejecting disturbances

Event 2: 4x4 Left Turns (200 points)

- ❖ autonomously drive in a 1m square (marked w/ tape) 4 times while balancing.
- ❖ 50 points for each loop where wheels do not cross the black tape. A full loop will count once the robot faces the same orientation it had once it began driving. I.e, you will need to face your starting orientation once you reach the final vertex.
- ❖ 25 points for each loop where wheels cross the tape
- ❖ Instructors will decide if a Balancebot is too far off the trajectory to be considered eligible for points.

Event 3: Straight Line Drag Racing (200 points)

- ❖ Race down the hallway, 11 meter (36ft), autonomously as fast as possible using odometry without falling over.
- ❖ Time will be measured from the instructors verbal "GO" command to the Balancebot stopping its forward momentum.
- ❖ You will have a 1 meter stop zone at the end, and must stop within the zone and stay balanced in this zone for 5 seconds.
- ❖ 50 points for finishing but failing to stop and balance within the end zone.
- ❖ 100 points for finishing and balancing
- ❖ +[100, 75, 50, 40, 30, 20, 10, 0] points for [1st - 8th] place

The rules of course for the next two events are as follows:

- 1) All gates start closed and you cannot pass through a closed gate.
- 2) Gates must be opened in the correct order.
- 3) Gates are opened by moving through them in the correct direction
- 4) Gates that are opened then stay open and can be passed through in either direction

Event 4: Manual Race Easy (200 points) ***ONLY 3 ATTEMPTS ALLOWED***

- ❖ Race through the course 3 times while a group member drives manually and never falls over or hits an obstacle -10 points for each obstacle hit.
- ❖ 100 points for finishing
- ❖ +[100, 75, 50, 40, 30, 20, 10, 0] points for [1st - 8th] place

Event 5: Manual Race Hard (200 points) ***ONLY 3 ATTEMPTS ALLOWED***

- ❖ Race through the course 3 times while a group member drives manually and never falls over, -10 points for each obstacle hit.

- ❖ 100 points for finishing
- ❖ +[100, 75, 50, 40, 30, 20, 10, 0] points for [1st - 8th] place

Checklist

Demonstrating basic functionality will be 25% of your grade. You will be given a checklist for instructors to sign off on. Keep this over the course of the project and scan it when completed. In order to ensure you are completing the lab on schedule, demonstrate the following to the instructors (in no particular order) **before the end of lab on Monday 2/03:**

- Demonstrate to an instructor your motors spinning using the test_motors program.
- Demonstrate to an instructor your functioning balance controller, even if it needs more tuning.
- Allow an instructor to drive your robot using the RC controller.
- Demonstrate a single lap of drive square.

Report Rubric

The lab report should be in IEEE format (a template is given below) and must be **no more than 8 pages**. Additional information can be placed in an appendix but will not be graded.

The requirements spreadsheet to be submitted with your report is [HERE](#).

Report Template

[Link to report layout](#)

Submission Instructions

Please zip and submit your project as given below. Points will be deducted for improper submission.

= team number as 2 digits

```
<#>-balancebot-w20.zip
- <#>-balancebot-w20/
  - report/
    - <#>-balancebot-w20.pdf
  - code/
    - <#>-balancebot-w20/
  - requirements.csv
  - checklist.png/pdf
  - signature.png/pdf
```

Certification and Peer Evaluation

[required for credit, 0 points]

Print or write the following on a sheet of paper:

"I participated and contributed to team discussions on each problem, and I attest to the integrity of each solution. Our team met as a group on [DATE(S)]."

Each team member should sign the paper (physically, not digitally), and a scan/photo of the paper should be included with your report. By doing so, you are asserting that you were materially involved in each answer, and that you certify that every solution complies with the collaboration policy of this course. In the event of a violation of the course's guidelines, your certification will be forwarded to the honor council.

If the statement requires qualification (i.e., it's not quite true), please describe the issue and the steps you took to mitigate it. If a team member did not materially contribute to a particular solution, then their score on that problem will generally be reduced. The staff will use their discretion based on the specific circumstances you identify, and we are happy to discuss potential problems prior to the problem set due date.

If your team works together, as required, this will be painless! :)

Each member of your team must also complete a peer evaluation, which will be posted after reports are due. Your evaluations of your teammates will remain private. If problems develop with your team, it is up to your team to resolve them: we will not intercede except in the most extreme situations.